

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

*I **HARI VISHNU N / 192524319** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Answer:

Introduction:

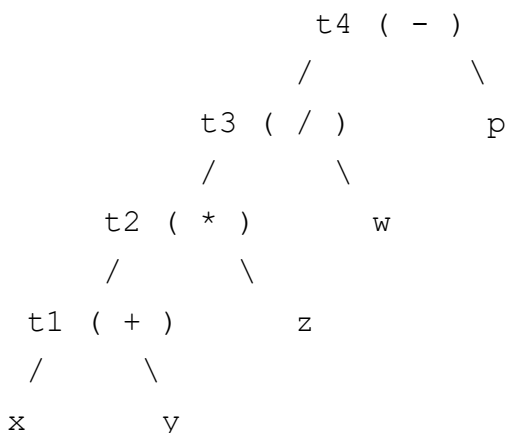
Three-address code (TAC) is an intermediate representation in which every statement contains at most one operator on the right-hand side and at most two operands, with the result stored in a temporary variable. It sits between the high-level source expression and the low-level target code, making it easy to analyze, optimize, and translate.

Step-by-step analysis of operations:

- $t1 = x + y \rightarrow$ computes the sum of x and y and stores it in temporary $t1$.
- $t2 = t1 * z \rightarrow$ multiplies the previously computed $t1$ with z and stores the result in $t2$.
- $t3 = t2 / w \rightarrow$ divides $t2$ by w and stores the result in $t3$.
- $t4 = t3 - p \rightarrow$ subtracts p from $t3$ to produce the final result $t4$.

Each instruction takes at most two operands and one operator, which is the defining property of three-address code (TAC). This makes the code easy to translate into an expression tree and, later, into machine code.

Equivalent expression tree (built bottom-up from the TAC):



Reading the tree bottom-up gives back the original expression: $t4 = (((x + y) * z) / w) - p$, confirming that the tree correctly preserves operator precedence and left-to-right evaluation order. Each internal node of the tree corresponds directly to one TAC instruction, and each leaf corresponds to a source-level operand (x, y, z, w, p).

Role of temporary variables:

- Each temporary (t1–t4) holds the result of one sub-expression, so the compiler never has to re-evaluate a partial result — the value is simply read back from the temporary when needed.
- Temporaries decompose a complex, nested expression into a linear sequence of simple operations, which maps naturally onto machine instructions that operate on at most two operands (matching real CPU instruction formats).
- Because every sub-result is named, the compiler can perform optimizations such as common sub-expression elimination, constant folding, and dead-code elimination directly on the temporaries without re-parsing the original expression.
- Temporaries make dependency analysis explicit: t2 depends on t1, t3 depends on t2, and t4 depends on t3 — this data-flow chain is exactly what the code generator uses for register allocation and instruction scheduling.
- They also enable reuse: if the same sub-expression appears again later in the program, the compiler can simply reference the existing temporary instead of recomputing it

Overall, using temporary variables converts a deeply nested arithmetic expression into a flat, easily analyzable sequence of instructions, which reduces repeated computation and makes subsequent optimization phases far more efficient.

Conclusion:

Temporary variables convert a deeply nested arithmetic expression into a flat, easily analyzable sequence of instructions. This reduces repeated computation, simplifies the construction of the expression tree, and gives later optimization phases a clean, linear structure to work on — making intermediate code generation both efficient and machine-independent.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Answer:

Introduction:

A symbol table is a data structure maintained by the compiler that stores information about every identifier in a program — its name, data type, size, scope, and memory location (offset). It is built during lexical/semantic analysis and consulted throughout compilation, particularly during memory allocation and code generation.

Step-by-step analysis of operations:

- $t1 = x + y \rightarrow$ computes the sum of x and y and stores it in temporary $t1$.
- $t2 = t1 * z \rightarrow$ multiplies the previously computed $t1$ with z and stores the result in $t2$.
- $t3 = t2 / w \rightarrow$ divides $t2$ by w and stores the result in $t3$.
- $t4 = t3 - p \rightarrow$ subtracts p from $t3$ to produce the final result $t4$.

How the compiler allocates memory:

- During semantic analysis, the compiler scans the symbol table and, for each variable, records its data type, size, and a memory offset relative to the start of the data segment (or activation record, for local variables).
- Allocation proceeds in the order variables are declared: p is placed first, followed by q , r , and s , with each new variable's offset computed as the previous offset plus the previous variable's size (adjusted for alignment).
- The size field in the symbol table (1, 8, 4, 4 bytes) tells the compiler exactly how many bytes of storage to reserve for each variable when generating the object code / activation record layout.
- The compiler also uses the data type to decide how the bit pattern in that memory region should be interpreted (e.g., IEEE-754 for float/double, two's complement for int).

Total memory required (raw, without padding):

Total = size(p) + size(q) + size(r) + size(s) = 1 + 8 + 4 + 4 = 17 bytes.

Effect of alignment and data types:

- Most architectures require multi-byte data types to start at an address that is a multiple of their own size (or a multiple of 4/8 bytes) for efficient memory access — this is called alignment.
- q (double, 8 bytes) needs 8-byte alignment. Since p (char) only occupies 1 byte, the compiler inserts 7 bytes of padding after p so that q starts at a properly aligned address.
- r (int, 4 bytes) and s (float, 4 bytes) each need 4-byte alignment; since q ends on an 8-byte (hence 4-byte) boundary, r and s can be placed immediately after one another without extra padding.
- Accounting for this padding, the compiler would typically reserve 1 (p) + 7 (padding) + 8 (q) + 4 (r) + 4 (s) = 24 bytes in the activation record, rather than the raw 17 bytes.

- This shows that memory organization depends not only on the sum of individual variable sizes but also on the data types involved and the padding the compiler inserts to satisfy hardware alignment constraints — trading a little extra space for faster memory access.

Conclusion:

Memory organization depends not only on the sum of individual variable sizes but also on the data types involved and the padding the compiler inserts to satisfy hardware alignment constraints. This is a direct trade-off between memory efficiency and access speed — a small amount of extra space (7 bytes here) is sacrificed so that the CPU can fetch multi-byte values in a single aligned memory access rather than multiple unaligned ones.

Q3. Optimization in Intermediate Code

Given the intermediate code:

$$t1 = m * n$$
$$t2 = p + q$$
$$t3 = m * n$$
$$t4 = t2 - t3$$

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Answer:

Introduction:

Code optimization is the phase of compilation that improves intermediate code to reduce execution time, memory usage, or code size, without changing the program's observable output. One of the most common and effective local optimizations is Common Sub-expression Elimination

Step-by-step trace:

- $t1 = m * n \rightarrow$ computes $m * n$ and stores it in $t1$.
- $t2 = p + q \rightarrow$ computes $p + q$ and stores it in $t2$ (independent of $t1$).
- $t3 = m * n \rightarrow$ computes $m * n$ again and stores it in $t3$ — this is exactly the same expression already computed for $t1$.
- $t4 = t2 - t3 \rightarrow$ subtracts $t3$ from $t2$ to get the final result.

Optimization opportunity identified — Common Sub-expression Elimination (CSE):

- Because m and n are not modified between the computation of $t1$ and $t3$, the expression $m * n$ in the third statement is a common sub-expression: it produces the same value as $t1$ and does not need to be recomputed.

- This redundancy is detected by comparing the operator and operands of each new instruction against previously computed expressions (often maintained in a value-table or DAG during code generation).
- The compiler can therefore eliminate the redundant multiplication and simply reuse $t1$ in place of $t3$.

Optimized intermediate code:

```
t1 = m * n
t2 = p + q
t4 = t2 - t1
```

How optimization improves execution efficiency:

- Removing the duplicate $t3 = m * n$ eliminates one multiplication instruction, directly reducing the number of CPU cycles spent on redundant computation.
- Fewer temporaries ($t3$ is no longer needed) reduces register/memory pressure, which can allow more variables to be kept in registers and lowers the chance of register spilling.
- On a larger scale, applying CSE consistently across a program can significantly shrink code size and execution time, especially inside loops where a redundant expression would otherwise be recomputed on every iteration.
- This illustrates the general principle of optimization: compute each distinct value only once, and reuse the stored result wherever the same value is needed again.

Conclusion:

This example illustrates the general principle of optimization: compute each distinct value only once, and reuse the stored result wherever the same value is needed again — improving both time and space efficiency of the generated code.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

```
if (x > y || p < q)
```

Intermediate code:

```
if x > y goto L1
if p < q goto L1
goto L2
L1: ...
```

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Answer:

Introduction:

Boolean expressions involving logical operators (&&, ||) are translated by the compiler not into explicit true/false-valued temporaries, but directly into a sequence of conditional and unconditional jumps. This "jumping code" approach is more efficient because it maps directly onto machine branch instructions and naturally supports short-circuit evaluation.

Interpreting the control flow:

- The condition $(x > y \parallel p < q)$ is true if either sub-condition is true, so the intermediate code is generated to jump to the "true" branch (L1) as soon as any one sub-condition succeeds.
- Instruction 1 (if $x > y$ goto L1): tests the first condition. If it is true, control jumps immediately to L1 (the true branch) without ever evaluating $p < q$.
- Instruction 2 (if $p < q$ goto L1): only reached when $x > y$ was false. It tests the second condition; if true, control again jumps to L1.
- Instruction 3 (goto L2): reached only if both conditions are false, transferring control to the false branch, L2.
- L1: ... marks the start of the code executed when the overall OR expression is true.

Short-circuit evaluation:

- Short-circuit evaluation means the second operand of a logical operator is evaluated only when necessary to determine the final truth value.
- For OR (||): if the first operand ($x > y$) is true, the whole expression must be true regardless of the second operand, so $p < q$ is skipped entirely.
- For AND (&&), the equivalent rule is the opposite:
- if the first operand is false, the whole expression must be false, so the second operand is skipped.
- This is exactly what the given intermediate code implements:
- the jump to L1 in the very first instruction bypasses the evaluation of $p < q$ whenever it is not needed.

Why this matters:

- It reduces the number of condition checks actually executed at run time, saving CPU cycles, especially when the second condition is expensive to evaluate (e.g., a function call or a loop).
- It allows safe patterns such as $(p \neq \text{NULL} \ \&\& \ p \rightarrow \text{value} > 0)$, where evaluating the second operand when the first is false would cause a runtime error — short-circuiting avoids this.
- It directly reflects how compilers translate Boolean expressions into efficient, jump-based control flow rather than computing a literal true/false value for every sub-expression, which would require extra temporaries and comparisons.

Conclusion:

This jump-based translation of Boolean expressions is both time-efficient (fewer checks) and

semantically safer (avoids evaluating unnecessary or unsafe conditions), which is why virtually all production compilers implement short-circuit evaluation this way.

Q5. Backpatching and Flow Control

Given:

Instruction	Target
200: if x==y goto _	(to be filled)
201: goto _	(to be filled)

Backpatch lists: truelist = {200}, falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

Answer:

Introduction:

Backpatching is a technique used during single-pass intermediate code generation for Boolean expressions and control statements. It solves the problem of generating jump instructions before their target labels are known.

Role of backpatching:

- When generating code for Boolean expressions and control statements in a single left-to-right pass, the compiler often does not yet know the target label of a jump instruction at the point the jump is generated (the label may correspond to code that appears later).
- Backpatching solves this by initially leaving the target field of such jump instructions blank (denoted "_") and recording the instruction's address in a list — truelist for jumps taken when the condition is true, and falselist for jumps taken when it is false.
- Once the compiler later determines the actual address the jumps should point to, it "patches" (fills in) the target field of every instruction in the corresponding list.

Demonstrating the update of target addresses:

- Instruction 200 (if x==y goto _) is added to truelist = {200} because it should jump to wherever the "true" code begins.
- Instruction 201 (goto _) is added to falselist = {201} because it should jump to wherever the "false" code begins.
- Suppose the true-branch code is later found to start at address 210. The compiler calls backpatch(truelist, 210), which fills in instruction 200 as: 200: if x==y goto 210.
- Similarly, if the false-branch code starts at address 215, backpatch(falselist, 215) fills in instruction 201 as: 201: goto 215.

- In this way, every instruction whose address was stored in a list gets its blank target replaced with the correct, now-known label — without needing to revisit or re-parse the source expression.

Importance in Boolean expression translation:

- Backpatching allows Boolean expressions and control statements (if, while, for, &&, ||) to be translated in a single pass, avoiding the need to generate an explicit syntax tree or make multiple passes over the code.
- It naturally supports short-circuit code generation, since the truelist/falselist of a sub-expression can be merged with those of surrounding expressions and patched together once the final target is known.
- It keeps the generated intermediate code compact and jump-based (rather than relying on explicit Boolean-valued temporaries), which maps efficiently onto real machine branch instructions.
- Overall, backpatching is a key technique that lets a compiler generate efficient control-flow code for nested and short-circuited Boolean expressions in one linear pass over the input.

Conclusion:

Backpatching is a key technique that lets a compiler generate efficient, jump-based control-flow code for nested and short-circuited Boolean expressions in one linear pass over the input — avoiding costly multi-pass translation while still producing correct target addresses.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand